

Object, Map, Set — Key-Value 저장과 중복 제거

코딩테스트에서 빈도 세기, 존재 여부 확인, 중복 제거에 핵심적으로 쓰이는 자료구조들.

위치: Object는 가장 기본적인 Key-Value 구조, Map/Set은 ES6에서 추가된 특화 구조

Object vs Map 비교

관점	Object	Map
키 타입	string/Symbol만 가능	어떤 타입이든 가능 (객체, 숫자 등)
순서	부분적 보장 (정수키 → 삽입순)	삽입 순서 완전 보장
크기 확인	<code>Object.keys(obj).length</code>	<code>map.size</code>
반복	<code>for...in, Object.entries()</code>	<code>for...of, map.forEach()</code>
코딩테스트 용도	빈도 세기 (가장 많이 사용)	키 타입이 중요할 때

Object 핵심 메서드

메서드/문법	설명	시간복잡도
<code>obj[key] / obj.key</code>	값 접근 (없으면 undefined)	O(1)
<code>obj[key] = value</code>	값 설정	O(1)
<code>Object.keys(obj)</code>	모든 키 배열 반환	O(n)
<code>Object.values(obj)</code>	모든 값 배열 반환	O(n)
<code>Object.entries(obj)</code>	[키, 값] 쌍의 배열 반환	O(n)
<code>delete obj[key]</code>	키-값 삭제	O(1)
<code>obj.hasOwnProperty(key)</code>	키 존재 여부	O(1)
<code>key in obj</code>	키 존재 여부 (프로토타입 체인 포함)	O(1)

Map 핵심 메서드

메서드	설명	시간복잡도
<code>map.set(key, value)</code>	키-값 저장	O(1)
<code>map.get(key)</code>	키로 값 조회 (없으면 undefined)	O(1)
<code>map.has(key)</code>	키 존재 여부	O(1)
<code>map.size</code>	저장된 쌍 개수	O(1)

메서드	설명	시간복잡도
<code>map.delete(key)</code>	키-값 삭제	$O(1)$
<code>map.entries()</code>	[키, 값] 이터레이터	$O(1)$
<code>map.forEach(callback)</code>	각 쌍에 대해 함수 실행	$O(n)$

Set — 중복 제거 특화

메서드	설명	시간복잡도
<code>set.add(value)</code>	값 추가 (이미 있으면 무시)	$O(1)$
<code>set.has(value)</code>	값 존재 여부	$O(1)$
<code>set.size</code>	요소 개수	$O(1)$
<code>set.delete(value)</code>	값 삭제	$O(1)$

핵심 패턴: **Array** → **Set** → **Array** 변환으로 중복 제거

```
const arr = [1, 2, 2, 3, 3, 3];
const unique = [...new Set(arr)];
console.log(unique); // [1, 2, 3]
```

패턴

코딩테스트 출제 빈도: 패턴1 빈도 세기 > 패턴2 고유 개수 > 패턴3 부분 문자열 검사 > 패턴1심화 빈도 + 조합

패턴 1: 빈도 세기 — Object vs Map

언제 쓰는가

- 각 요소가 몇 번 등장하는지 카운트할 때
- 코딩테스트에서 가장 자주 나오는 Object 활용 패턴
- **값(value)에 의미가 있는 패턴** — 값이 등장 횟수를 나타냄

실무에서는

- 로그 집계, 투표 결과 집계, 장바구니 수량 관리

```
const fruits = ["apple", "banana", "apple", "cherry", "banana", "apple"];

// === 방법 1: Object ===
// 직관적이고 간결. 코딩테스트에서 가장 많이 쓰는 방식.
const count1 = {};
```

```

for (const fruit of fruits) {
  count1[fruit] = (count1[fruit] || 0) + 1;
}
console.log(count1); // { apple: 3, banana: 2, cherry: 1 }

// === 방법 2: Map ===
// 키 타입이 문자열 외(숫자, 객체 등)일 때 필요.
const count2 = new Map();
for (const fruit of fruits) {
  count2.set(fruit, (count2.get(fruit) || 0) + 1);
}
console.log(count2); // Map { 'apple' => 3, 'banana' => 2, 'cherry' => 1 }

```

방법 선택 기준

- 키가 문자열이면 **Object**가 간결 (코테에서는 대부분 이 경우)
- 키가 숫자, 객체 등 문자열 외 타입이면 **Map** 필요

```

// 키가 문자열 → Object가 간결
const obj = {};
obj["apple"] = 1;

// 키가 객체 → Map 필요 (Object는 키를 문자열로 변환해버림)
const map = new Map();
map.set({id: 1}, "value");

```

가장 많이 나온 요소 찾기

```

const count = {};
for (const fruit of fruits) {
  count[fruit] = (count[fruit] || 0) + 1;
}

let maxFruit = "";
let maxCount = 0;
for (const [fruit, cnt] of Object.entries(count)) {
  if (cnt > maxCount) {
    maxCount = cnt;
    maxFruit = fruit;
  }
}
console.log(`${maxFruit}: ${maxCount}번`); // apple: 3번

```

심화: 빈도 + 그룹별 조합 계산

언제 쓰는가

- 그룹별로 분류한 뒤 조합 수를 계산할 때
- 패턴 1로 빈도를 센 뒤, 그 결과를 활용하는 패턴

실무에서는

- 상품 옵션 조합, 필터 조합

```
// 의상 조합 문제: 종류별로 1개씩 선택 (아무것도 안 입는 경우 제외)
const clothes = [
  ["yellow_hat", "headgear"],
  ["blue_sunglasses", "eyewear"],
  ["green_turban", "headgear"],
];

// 종류별 개수 세기 (패턴 1)
const typeCount = {};
for (const [name, type] of clothes) {
  typeCount[type] = (typeCount[type] || 0) + 1;
}

console.log(typeCount); // { headgear: 2, eyewear: 1 }

// 조합 수 계산: 각 종류별 (선택 수 + 안 입는 경우 1) 곱한 후 - 전부 안 입는 경우 1
let combinations = 1;
for (const [type, cnt] of Object.entries(typeCount)) {
  combinations *= (cnt + 1); // headgear: 3가지(2개 + 안입기), eyewear: 2가지
  (1개 + 안입기)
}
combinations -= 1; // 전부 안 입는 경우 제외

console.log("조합 수:", combinations); // 조합 수: 5
// yellow_hat, green_turban, blue_sunglasses,
// yellow_hat+blue_sunglasses, green_turban+blue_sunglasses
```

패턴 2: 중복 제거 + 고유 개수 — Set

언제 쓰는가

- 서로 다른 종류가 몇 개인지 셀 때
- "종류의 수"를 구하는 문제
- **키(key)에 의미가 있는 패턴** — 키의 존재 자체가 "이 종류가 있다"를 나타냄 (패턴 1은 값에 의미가 있었다면, 패턴 2는 키에 의미가 있음)

실무에서는

- 고유 방문자 수, 태그 중복 제거

```
const animals = ["cat", "dog", "cat", "bird", "dog", "fish"];

// 기본: 고유 개수만 필요하면 Set
const uniqueSet = new Set(animals);
console.log("종류 수:", uniqueSet.size); // 종류 수: 4

// 중복 제거된 배열
const uniqueArr = [...uniqueSet];
console.log(uniqueArr); // ["cat", "dog", "bird", "fish"]

// 심화: 각 종류별 개수까지 필요하면 Object로 빈도를 세고 키 개수
const countObj = {};
for (const animal of animals) {
  countObj[animal] = (countObj[animal] || 0) + 1;
}
console.log("종류 수:", Object.keys(countObj).length); // 종류 수: 4
console.log("각 종류별 개수:", countObj); // { cat: 2, dog: 2, bird: 1, fish: 1 }
```

포켓몬 문제 패턴: N/2 선택 시 최대 종류 수

```
function maxTypes(nums) {
  const types = new Set(nums).size;
  const pick = nums.length / 2;
  return Math.min(types, pick);
}

console.log(maxTypes([3, 1, 2, 3])); // 2 (종류 3, 선택 2 → 2)
console.log(maxTypes([3, 3, 3, 2, 2, 4])); // 3 (종류 3, 선택 3 → 3)
console.log(maxTypes([3, 3, 3, 2, 2, 2])); // 2 (종류 2, 선택 3 → 2)
```

패턴 3: 부분 문자열 검사 — substring + 해시 O(1) 조회

언제 쓰는가

- 한 문자열이 다른 문자열의 부분 문자열(접두사, 접미사, 내부)인지 확인할 때
- 전화번호부 문제 등

이 패턴에서 Object/Set을 쓰는 이유

핵심은 키의 존재 여부를 **O(1)**로 빠르게 확인하기 위해서다. 배열의 `includes()`는 O(n)이지만, Object의 `obj[key]`나 Set의 `set.has()`는 O(1)이다.

이 예제처럼 값(value)이 필요 없고 존재 여부만 확인하면 **Set이 더 적절**하다. 값에 부가 정보를 담아야 할 때 (예: 전화번호 → 사용자 이름) Object를 쓰면 된다.

```
// 전화번호부에서 어떤 번호가 다른 번호의 접두사인지 확인
const phoneBook = ["119", "97674223", "1195524421"];

// 존재 여부만 확인하므로 Set이 더 적절
const set = new Set(phoneBook);

let hasPrefix = false;
for (const phone of phoneBook) {
  // 한 글자씩 늘려가며 접두사가 존재하는지 확인
  for (let i = 1; i < phone.length; i++) {
    const prefix = phone.substring(0, i);
    if (set.has(prefix)) {
      console.log(`${prefix}는 ${phone}의 접두사입니다`);
      // 119는 1195524421의 접두사입니다
      hasPrefix = true;
    }
  }
}

console.log("접두사 존재:", hasPrefix); // 접두사 존재: true
```

접두사 외에도 같은 방식으로 응용 가능

```
const phone = "1195524421";

// 접두사: 앞에서부터 자름
phone.substring(0, i); // "1", "11", "119", ...

// 접미사: 뒤에서부터 자름
phone.substring(phone.length - i); // "1", "21", "421", ...

// 내부 문자열: 모든 구간을 자름
phone.substring(i, j); // 가능한 모든 부분 문자열
```